

This looks like a **Senior GenAI Data Scientist (L'Oréal) 60-minute interview round** focused on **LLMs, RAG, Agentic AI, FastAPI, Infrastructure, Optimization, and Production AI Systems**.

I'll enhance the answers to an **interview-ready level** so you can explain both conceptually and practically.

1. What is Async Programming?

Async programming allows a program to execute other tasks while waiting for I/O operations such as:

- API Calls
- Database Queries
- LLM Calls
- File Reads/Writes

Instead of blocking the thread, control is returned to the event loop.

Why useful in GenAI?

Suppose:

```
1 llm.invoke(prompt)
```

takes 5 seconds.

If 100 users hit the endpoint simultaneously:

- Sync → 100 requests wait sequentially
 - Async → Requests handled concurrently
-

FastAPI Async Endpoint

Single Request

```
1 from fastapi import FastAPI
2
3 app = FastAPI()
4
5 @app.post("/generate")
6 async def generate(prompt: str):
7
8     response = await llm.ainvoke(prompt)
```

```
9
10     return {"answer": response}
```

Multiple Prompts

```
1  import asyncio
2
3  @app.post("/batch_generate")
4
5  async def batch_generate(prompts: list[str]):
6
7      tasks = [
8          llm.ainvoke(prompt)
9          for prompt in prompts
10     ]
11
12     responses = await asyncio.gather(*tasks)
13
14     return responses
```

2. Multithreading vs Parallel Processing

Multithreading

Multiple threads within the same process.

Best for:

- API Calls
- Database Calls
- Network Requests
- RAG retrieval

Example:

```
1  Thread1 -> Call VectorDB
2  Thread2 -> Call LLM
3  Thread3 -> Call Metadata Service
```

Parallel Processing

Uses multiple CPU cores.

Best for:

- Embedding generation
 - OCR processing
 - Large document parsing
 - Model training
-

Pseudocode

Multithreading

```
1  from concurrent.futures import ThreadPoolExecutor
2
3  def retrieve_docs():
4      pass
5
6  def run_guardrail():
7      pass
8
9  with ThreadPoolExecutor() as executor:
10
11     future1 = executor.submit(retrieve_docs)
12     future2 = executor.submit(run_guardrail)
13
14     result1 = future1.result()
15     result2 = future2.result()
```

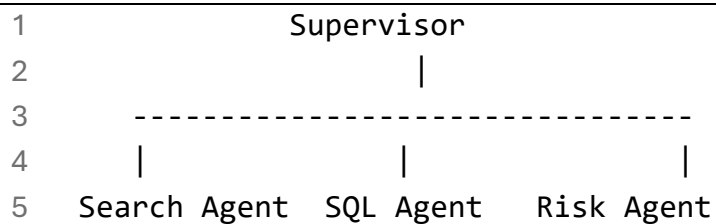
Multiprocessing

```
1  from multiprocessing import Pool
2
3  def generate_embedding(chunk):
4      pass
5
6  with Pool(processes=8) as pool:
7
8     embeddings = pool.map(
```

```
9         generate_embedding,  
10         chunks  
11     )
```

3. How Agents Communicate in Supervisor Framework?

In a supervisor architecture:



Supervisor:

- Receives query
- Decides agent
- Routes task
- Aggregates response

Communication Patterns

Message Passing

```
1 {  
2   "task":"retrieve customer risk",  
3   "customer_id":"123"  
4 }
```

Shared State

All agents update:

```
1 state = {  
2   "query": "",  
3   "retrieval_result": "",  
4   "risk_score": ""  
5 }
```

LangGraph Example

```
1 StateGraph()  
2  
3 Supervisor Node  
4  
5 -> Search Agent  
6  
7 -> SQL Agent  
8  
9 -> Risk Agent  
10  
11 -> Final Node
```

Each node updates shared state.

LlamaStack

Typically implemented using:

```
1 Agent  
2 Tool  
3 Memory  
4 Session  
5 Workflow
```

Supervisor delegates work to specialized agents through workflows.

4. LLM Security & Governance

Production AI requires multiple layers.

Prompt Injection Protection

Detect:

```
1 Ignore previous instruction  
2 Reveal system prompt
```

Use:

- Llama Guard
- Prompt Shield

- NeMo Guardrails
-

PII Detection

Mask:

- | | |
|---|--------------|
| 1 | Phone Number |
| 2 | Aadhaar |
| 3 | Passport |
| 4 | Email |

Example:

- | | |
|---|-----------------------|
| 1 | John -> <PERSON> |
| 2 | 9999999999 -> <PHONE> |
-

RBAC

Role-based access

- | | |
|---|----------------------------|
| 1 | Underwriter -> Access |
| 2 | Customer -> Limited Access |
-

Output Moderation

Detect:

- Toxicity
 - Hallucination
 - Data Leakage
-

Governance

Track:

- | | |
|---|----------|
| 1 | User |
| 2 | Prompt |
| 3 | Response |
| 4 | Model |
| 5 | Version |
| 6 | Latency |
| 7 | Cost |

Store in:

- | | |
|---|-----------|
| 1 | LangSmith |
| 2 | MLFlow |
| 3 | Phoenix |
| 4 | Datadog |

5. How do you Evaluate LLM in RAGAS?

RAGAS evaluates RAG systems without requiring human labels.

Common metrics:

Faithfulness

Answer supported by retrieved context?

- | | |
|---|-------------------------|
| 1 | High = No hallucination |
|---|-------------------------|

Answer Relevancy

Did answer address query?

Context Precision

How much retrieved information is relevant?

Context Recall

Did retrieval capture all necessary context?

Precision@K

Measures relevance in top-K retrieved chunks.

Formula:

- | | |
|---|--|
| 1 | $\text{Precision@K} =$ |
| 2 | $\frac{\text{Relevant Retrieved Chunks}}{K}$ |

```
3 -----
4 Total Retrieved Chunks
```

Example:

```
1 Top 5 chunks
2 3 relevant
3
4 Precision@5 = 3/5 = 0.6
```

Recall@K

Measures retrieval coverage.

```
1 Recall@K =
2 Relevant Retrieved Chunks
3 -----
4 Total Relevant Chunks
```

Example:

```
1 Relevant chunks available = 10
2
3 Retrieved = 8
4
5 Recall@K =
6 8/10 = 0.8
```

6. What is MMR and RRF?

MMR (Maximum Marginal Relevance)

Balances:

- Relevance
- Diversity

Without MMR:

```
1 Chunk1: Policy
2 Chunk2: Same Policy
3 Chunk3: Same Policy
```

With MMR:

```
1 Chunk1: Policy
```


- | | |
|---|----------------|
| 2 | Chunk2: Claims |
| 3 | Chunk3: Risk |

Reduces duplication.

RRF (Reciprocal Rank Fusion)

Combines rankings from multiple retrievers.

Example:

BM25

- | | | |
|---|------|-------|
| 1 | Doc1 | Rank1 |
| 2 | Doc2 | Rank2 |

Vector Search

- | | | |
|---|------|-------|
| 1 | Doc2 | Rank1 |
| 2 | Doc3 | Rank2 |

RRF combines both scores.

Formula:

- | | |
|---|----------------|
| 1 | RRF = |
| 2 | $\sum 1/(k+r)$ |

Very common in Hybrid Search.

7. KV Caching & Speculative Decoding

KV Cache

During transformer inference:

Each generated token creates:

- | | |
|---|-------|
| 1 | Key |
| 2 | Value |

Store them instead of recomputing.

Without cache:

- | | |
|---|-----------------------------|
| 1 | Recalculate entire sequence |
|---|-----------------------------|

With cache:

- | | |
|---|---------------------------------|
| 1 | Reuse previous attention states |
|---|---------------------------------|

Benefit:

- 1 Lower Latency
- 2 Higher Throughput

Speculative Decoding

Uses:

- 1 Small Draft Model
- 2 +
- 3 Large Target Model

Process:

- 1 Small Model predicts:
- 2 A B C D
- 3
- 4 Large Model verifies.

Accepted tokens returned immediately.

Benefits:

- 1 2x-4x Faster Generation

8. Cost Optimization in GenAI

Interviewers love this question.

Reduce Token Usage

Compress prompts.

- 1 5000 tokens -> 1500 tokens

Better Chunking

Retrieve:

- 1 Top-3
- 2 instead of
- 3 Top-20

Model Routing

Simple query:

```
1 GPT-4o-mini
```

Complex query:

```
1 GPT-4o
```

Response Caching

Cache:

```
1 Frequently Asked Questions
```

Use Redis.

Quantization

```
1 FP16
2 INT8
3 INT4
```

Reduces GPU cost.

Batch Inference

```
1 100 prompts
2 →
3 Single API call
```

9. Difference Between GET and POST

GET

Retrieve data.

```
1 GET /customer?id=123
```

Characteristics:

- Safe
- Idempotent
- Query parameters visible

POST

Create or process data.

```
1 POST /generate
```

Body:

```
1 {  
2   "prompt": "Explain insurance policy"  
3 }
```

In GenAI

Use POST because:

- ```
1 Prompt can be large
2 Contains sensitive data
```

---

## 10. Chunking Strategies

### Lazy Chunking

Chunk only when required.

Saves preprocessing cost.

---

### Semantic Chunking

Split using meaning boundaries.

- ```
1 Paragraph A  
2 Paragraph B  
3 Topic Change
```

Better retrieval quality.

Hybrid Chunking

Combines:

- 1 Structural
- 2 +
- 3 Semantic

Example:

- 1 Heading Based
- 2 +
- 3 Embedding Based

Why Recursive Character Splitter?

Most used because:

- 1 Section
- 2 -> Paragraph
- 3 -> Sentence
- 4 -> Word

Attempts splitting hierarchically.

- 1 ["\n\n", "\n", ".", " "]

Preserves context.

Chunk Overlap

Overlap prevents context loss.

Example:

- 1 Chunk1:
- 2 Policy starts on Jan 1
- 3
- 4 Chunk2:
- 5 Jan 1 and expires on Dec 31

Missing overlap causes context break.

Typical Values

- 1 Chunk Size = 512-1000 tokens
- 2
- 3 Overlap = 10%-20%

Start:

- 1 800 chunk
- 2 150 overlap

Then evaluate using RAGAS.

11. What Do You Know About LLM-D?

LLM-D (Large Language Model Distributed Inference) is an emerging architecture focused on:

- 1 Disaggregated Inference
- 2 KV Cache Sharing
- 3 Intelligent Routing
- 4 Distributed Serving

Goals:

- Higher throughput
- Lower latency
- Better GPU utilization

Commonly discussed with:

- 1 vLLM
- 2 Ray
- 3 Kubernetes
- 4 NVIDIA Dynamo

Designed for large-scale enterprise deployments.

12. RAG Pipeline is Slow. First Step of Inspection?

My first step:

Break latency across stages.

- 1 User Query
- 2 |
- 3 Retriever
- 4 |
- 5 Reranker
- 6 |
- 7 LLM

8	
9	Response

Measure:

1	Retrieval Latency
2	Embedding Latency
3	Reranking Latency
4	Inference Latency

Typically:

70%-90% latency lies in:

1	LLM Inference
---	---------------

Then I'd investigate:

1. Retrieval time
2. Vector DB time
3. Reranker time
4. Token count
5. Model size
6. GPU utilization

Tools:

1	LangSmith
2	Phoenix
3	OpenTelemetry
4	Datadog

13. Design an Underwriting Chat Assistant (Complex PDFs)

This is likely the strongest system-design question.

Problem

Input:

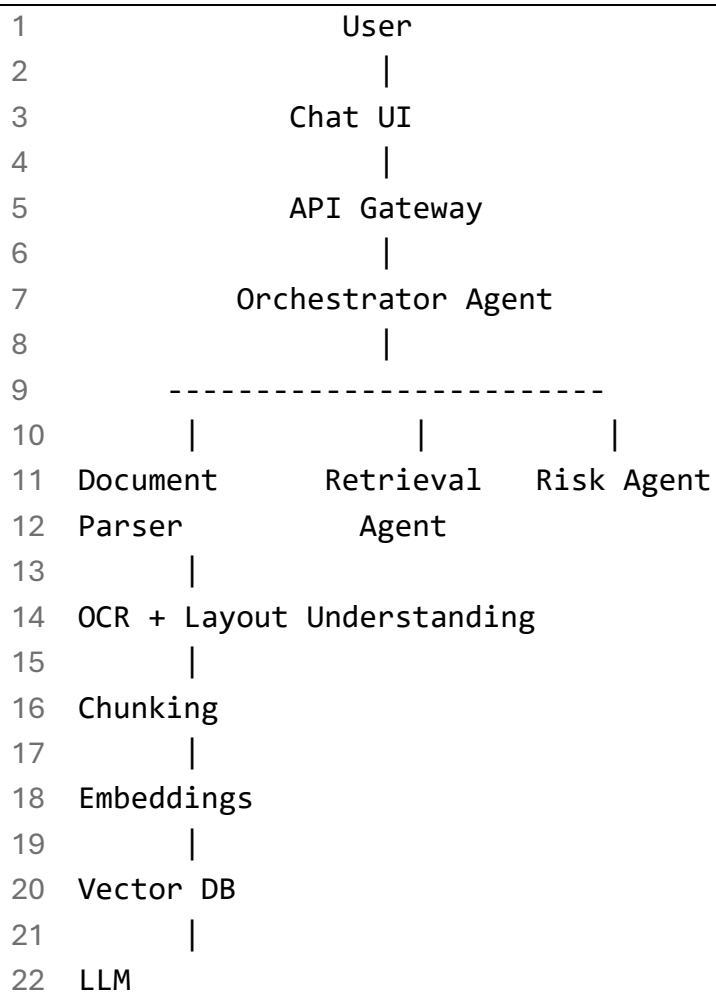
1	Insurance Manuals
2	Policy PDFs
3	Risk Documents
4	Medical Reports

- 5 Scanned Documents
- 6 Tables
- 7 Images
- 8 Nested Grids

Need:

- 1 Chat Interface
- 2 Risk Analysis
- 3 Policy QA
- 4 Underwriting Recommendation

Architecture



Document Processing

Simple PDF extraction fails.

Use:

- 1 Azure Document Intelligence
- 2 Unstructured.io
- 3 LLamaParse
- 4 Docling

Extract:

- 1 Tables
- 2 Nested tables
- 3 Images
- 4 Headers
- 5 Footers
- 6 Captions
- 7 Forms

Multimodal Parsing

For images:

- 1 Medical scans
- 2 Risk charts
- 3 Handwritten notes

Use:

- 1 GPT-4o
- 2 Gemini
- 3 Claude

Generate image captions.

Advanced Chunking

Store metadata:

```
1 {  
2   "document":"policy.pdf",  
3   "page":22,  
4   "section":"Exclusions",  
5   "table_id":"T4"  
6 }
```

Use:

- 1 Semantic + Hierarchical Chunking
-

Hybrid Retrieval

Combine:

- | | |
|---|------------------------|
| 1 | BM25 |
| 2 | + |
| 3 | Vector Search |
| 4 | + |
| 5 | RRF |
| 6 | + |
| 7 | Cross Encoder Reranker |

This gives highest recall for underwriting documents.

Underwriting Agents

Policy Agent

Understands policy wording.

Medical Agent

Reads health records.

Risk Agent

Calculates underwriting risk.

Compliance Agent

Ensures regulations.

Security

- PII masking
 - Audit trail
 - Encryption
 - RBAC
 - Prompt-attack detection
-

Evaluation

- | | |
|---|--------------|
| 1 | RAGAS |
| 2 | Faithfulness |

- | | |
|---|--------------------|
| 3 | Context Precision |
| 4 | Latency |
| 5 | Citation Accuracy |
| 6 | Hallucination Rate |